

A Parametric Design of Globals and Capabilities

Fengyun Liu

TypeScope

Clément Blaudeau

Cryspen

Ondřej Lhoták

University of Waterloo

Abstract

It is widely known that global mutable state introduced by global variables compromises modular reasoning and leads to concurrency issues. However, the convenience of sharing information among different program parts is so appealing that global variables make it into almost all languages, even recent languages such as Go and Rust.

We propose *context parameters* as an alternative design to global variables. Context parameters provide the convenience of global variables without sacrificing *parameterization* of programs. Context parameters can be rebound to new values in a context as desired — they are just parameters without the syntactic overhead of function parameters. Meanwhile, a software architect can strictly control which parts of a program may have access to which context parameter. Submodules are unable to secretly introduce global state without the software architect knowing about it at the top level. The control is enforced statically by the compiler.

We show that *context parameters* have the potential to address several real-world programming problems, such as contextual abstraction, scalability of optional parameters, and dependency injection. Most importantly, they facilitate the design of object-capability systems by eliminating ambient authorities, solving the usability problem of passing authorities and enabling static control of authorities.

2012 ACM Subject Classification Software and its engineering → Object oriented languages; Software and its engineering → Classes and objects

Keywords and phrases global variables, context parameters, parametricity

1 Introduction

Language designers have a moral obligation to create languages that facilitate the construction of more reliable programs, as famously put by Hoare [3]:

An unreliable programming language generating unreliable programs constitutes a far greater risk to our environment and to our society than unsafe cars, toxic pesticides, or accidents at nuclear power stations. Be vigilant to reduce that risk, not to increase it.

One language feature that is widely considered as problematic is *global variables*, as famously criticized by Wulf and Shaw [16] about 50 years ago. Programmers usually use global variables to share information among different program parts, either for the whole program or at a module level.

The convenience of sharing information among different program parts is so appealing that global variables appear in almost all languages, including recent ones such as Go and Rust. Neither functional programming languages nor object-oriented programming languages manage to escape from global variables. In OCaml, they appear as module variables. In Java, they take the form of static fields. In Scala, they shelter inside global objects.

1.1 Global Variables, Modularity, Security and Parameterization

We identify two key issues related to global variables: modularity and parameterization. Given that the former is well-known, we will highlight the latter, which is less discussed.

Problem of uncontrolled global state

Global variables hinder modularity. Through global variables, submodules can easily introduce global state, both in the form of a mutable global variable or indirectly through an immutable variable pointing to mutable objects. Such global state compromises modular reasoning – as the behavior of a submodule depends on the whole program – and is prone to concurrency issues. Moreover, as programs grow big, it becomes increasingly harder to ensure that the use of the global variables validates the right invariants, and changing those invariants requires problematic program-wide updates [1, 5].

In the context of security, uncontrolled global state is a nightmare in safely composing and confining untrusted code, preventing leak of authorities and performing security auditing [4, 8].

Yet experience shows that some program designs naturally rely on forms of global state. However, introduction of global state should be an explicit decision of the software architect at the top level of a program instead of at a module level. As programming language researchers, we indirectly owe software architects a debt in providing a mechanism to control global state of programs at the top level of a program.

Problem of parameterization

Another equally important but less discussed problem related to global variables is *parameterization*. If more behaviors of a software module can be controlled from outside by parameters, such a module will be easier to test and reuse. We say that such a software module has high parameterization. The problem is how to achieve high parameterization of software modules with low cost.

The most common way to achieve parameterization is to use function parameters. However, function parameters fall short in achieving high parameterization of software modules because they incur significant syntactic overhead. First, function parameters contaminate the signature of functions in the call chain. Second, in the presence of a large number of parameters, programmers usually need to pay the price of packing/unpacking parameters.

Classes in object-oriented programming that share a batch of parameters among methods can be seen as a way to mitigate the problem of parameterization. However, classes do not address the problem from the root: (1) the constructor of the class still needs to take a large number of parameters as input; (2) the parameters still need to propagate from the call chain to the creation of the class instance. (3) classes do not help in handling the problem where the parameters do not share the same life interval.

Intimidated by the difficulty of achieving parameterization, programmers often resort to using global variables for things that they deem are *infrequently changed*. This can be illustrated by the following Java example which implements a simple concurrent task queue service:

```

84
85 1 class TaskQueue {
86 2     static int THREADS = 10
87 3     static ExecutorService executorService = createThreadPool(THREADS);
88 4
89 5     static Future<T> <T> schedule(Callable task) {
90 6         return scheduleWithRetry(task, 0);
91 7     }
92 8     static Future<T> <T> scheduleWithRetry(Callable task, int retry) { ... }
93 9 }
94

```

In the first prototype of the code, it is reasonable for the programmer to assume that the `executorService` will rarely be changed. Therefore, it is reasonable to store it in a static field. Consequently, the class `TaskQueue` will be less testable and reusable due to the reduced parameterization: it is impossible to provide an executor service from outside. In particular, in testing, we would like to use a sequential executor to check that the retry logic is correct.

In the above, we see that global variables serve as an escape hatch for things that are *infrequently changed*, as parameterization is too expensive. We can imagine that many such decisions will accumulate in prototyping to the state where it will be painful to make changes later. Programmers deserve something better for prototyping and parameterization that would not degrade testability and reusability of programs.

As we will see, common real-world programming problems, such as contextual abstraction, dependency injection, scalable optional parameters, and software evolution, are just instances of the problem of parameterization (Section 4).

So the problem is not that global variables are bad, but that we need alternatives to replace global variables and uphold parameterization. However, more than 50 years after Wulf and Shaw [16] published *Global variable considered harmful*, there are still no alternatives in the toolbox of language designers that enable them to better support parameterization while satisfying programmers' taste for global variables.

Wulf and Shaw [16] mentioned that a suitable alternative design would need to satisfy the following requirements:

1. By default, a non-local variable should not be visible in the nested scope.
2. The permission to use a name should be a contract between creator and user.
3. The permission to access a structure and its sub-structures should be decoupled.
4. It should be possible to distinguish different types of accesses.
5. The declaration, access and binding of a non-local variable should be decoupled.

In the above, requirements (3) and (4) can be enforced with information hiding mechanisms in object-oriented programming. The other requirements are all about access control and explicit contracts.

With the design requirements in mind, we tackle the following language design problem:
Can we have an alternative language design that eliminates global variables in the name of parameterization while keeping their benefits?

1.2 Context Parameters

We answer the question above affirmatively with a new and simple language feature which we call *context parameters* (Section 2). Context parameters are based on a reflection on names in programming languages.

Generally, a name needs to be bound before it can be used. If we record the stack depth at runtime, we can compute the *stack distance* between the binding time of a name and its usage time:

$$\text{binding} \longleftrightarrow \text{usage}$$

For traditional function parameters, the stack distance is very short, which we call *close binding*. This is because function parameters have to be bound at call-sites immediately before the control flow transfers to the function and they may only be used inside the function body.

For function parameters, the close binding provides a convenient way to ensure that a parameter is always bound before it is used. However, the rigid requirement of close binding implies a parameter may not be used in a deeper call context. This is the root cause why function parameters incur syntactic overhead in achieving high parameterization of software modules.

Context parameters introduce a different kind of parameter that enables *remote binding* — the stack distance between binding and usage can be large. Remote binding enables usage of the binding for a parameter in a deep call context. The major features of context parameters are the following:

1. They are globally declared and lexically scoped like global variables;
2. They are parameters that always pass from upstream binding sites to downstream usage sites implicitly in the call chain;
3. As parameters, they can be rebound to a different value as desired in a nested context without affecting the outer calling context;
4. They may be declared with a default value expression which will be used if no binding is provided for the optional context parameter at places of its usage;
5. The upstream of a call chain can statically control what context parameters are allowed to pass through into submodules.

We propose context parameters as an alternative design to global variables, as they enjoy the convenience of the latter without sacrificing parameterization. We make this vision concrete by implementing a new language that replaces global variables with context parameters and demonstrate its feasibility and usability based on small examples (Section 2 and 4).

Context parameters bear some similarity to dynamic scoping. We discuss the subtle differences in related work (Section 6).

1.3 Contributions

More concretely, our paper makes the following contributions:

1. We analyze the problem of parameterization and propose context parameters as a simple and convenient mechanism to achieve parameterization of software modules. We show that context parameters can serve as an alternative design to global variables — they enjoy the convenience of global variables without sacrificing parameterization and modularity (Section 2).
2. We propose a type system which can ensure the safe usage of context parameters at runtime. The type system also affords a solution to the *deep capture problem* in the presence of first-class functions. Moreover, it provides a mechanism for the upstream of a call chain to statically control what context parameters may be passed downstream to submodules (Section 3). We prove soundness of the type system (Appendix A).

- 171 3. We show that context parameters are a flexible tool that lends itself to addressing common
172 programming problems, such as contextual abstraction, scalability of optional parameters,
173 dependency injection, and software evolution. These problems are in essence instances of
174 the problem of parameterization (Section 4).
- 175 4. We show that context parameters advance and facilitate the design of object-capability
176 systems by eliminating ambient authorities, supporting attenuation of authorities, solving
177 the usability problem of passing authorities and enabling static control of authorities
178 (Section 5).
- 179 5. We make the design vision concrete by implementing a new programming language that
180 eliminates global variables with context parameters. We demonstrate feasibility and
181 usability of the language with example programs (Section 2 and 4).

182 2 A Taste of Context Parameters

183 This section introduces context parameters informally with small examples and conveys their
184 semantics and safety guarantee intuitively.

185 Language model

186 A program in our language has only 2nd-class functions and context parameters at the top
187 level.¹ In particular, *there are no global variables*. It supports common data types (e.g.,
188 string, integer, option types), control flow (**if/else**, **while**), first-class functions (a.k.a.
189 lambdas), ADTs (algebraic data types) and functions for performing I/O.

190 2.1 Context Parameters and Remote Binding

191 The following is an example illustrating the usage of context parameters:

```
192 1 param lang: String
193 2
194 3 def hello(name: String): String =
195 4   if lang == "fr" then "Bonjour, " + name
196 5   else "Hello, " + name
197 6
198 7 def processRequest(userId: Int) =
199 8   ... hello(name) ...
200 9
201 10 processRequest(userId) with lang = getUserLang(userId)
```

204 In the code above, we define a context parameter `lang`, which is used in the function
205 `hello`. The parameter is provided from an outer context when the processing of a user
206 request starts. The binding is only effective during processing of the request.

207 One way to think about context parameters is to compare them with function parameters.
208 In terms of functionality, context parameters are similar to function parameters — they serve
209 as extra input to functions. The main difference is that context parameters are *declared at*
210 *top-level, bound remotely and can be used deep in the calling context*.

211 The binding for a context parameter can be *shadowed* in a nested calling context:

¹ In our language, the top-level names may be defined in different namespaces in different files. However, for the sake of presentation, the example does not use such features.

```

212
213 1 param fontSize: Int
214 2
215 3 def renderText(text: String) =
216 4   ... fontSize ...
217 5
218 6 def renderH2(h2: H2Element) =
219 7   renderText(h2.text) with fontSize = 20 // shadowing, fontSize = 20
220 8
221 9 def renderElement(e: Element) =
222 10  ...renderText(p.text)      // fontSize = 14
223 11  ...renderH2(h2)           // fontSize = 14
224 12  ...renderText(label.text) // fontSize = 14
225 13
226 14 def renderDiv(div: DivElement) = ...renderElement e with fontSize = 14
227

```

In the example above, we pass the font size 14 for rendering an element in a `div` block (line 14). The first call to `renderText` will use the font size 14 (line 10). However, in the call to `renderH2`, the value of `fontSize` is rebound to 20 (line 11 and 7). In this case, the outer binding is shadowed by the inner binding. Finally, the last call `renderText(label.text)` will use the font size 14 bound from its enclosing context (line 12).

2.2 Static Check: Binding Before Usage

Context parameters are equipped with a type system which ensures that a context parameter is always bound before it is used. Given the following program:

```

236
237 1 param newLine: String
238 2
239 3 def foo() =
240 4   print
241 5     "Hello, world!" + newLine
242 6
243 7 def bar() = foo
244 8
245 9 def main = bar
246

```

The compiler produces the following error:

```

248 ----- Error at hello.jo:9:5 -----
249 | def main = bar
250 |   ^^^
251 |   Context parameter not provided: newLine
252
253 The following is the trace that leads to the problem:
254 |— def main = bar [ hello.jo:9:12 ]
255 |   ^^^
256 |— def bar() = foo [ hello.jo:7:13 ]
257 |   ^^^
258 |   "Hello, world!" + newLine [ hello.jo:5:23 ]
259 |     ^^^^^

```

We will detail in Section 3 the type system that the compiler employs to ensure the safe usage of context parameters.

2.3 Optional Context Parameters

Suppose we are developing the data access layer of a database application. For the API to search products by a keyword, we would like to limit the number of returned results. It is natural to model the parameter as an *optional* context parameter:

```

1 param maxResultCount: Int = 100 // maximum result count
2
3 def search(keyword: String) =
4   ... maxResultCount ...

```

In the example above, if the caller of `search` does not provide a binding for `maxResultCount`, the default value 100 will be used. Otherwise, the supplied value will be effective.

The default values of optional context parameters are usually constants. However, they can be any expressions that produce a value of the specified type, as long as the evaluation of the expression does not depend on any context parameters from outside.²

Semantically, an optional context parameter declaration below

```

1 param name: T = rhs

```

will desugar to:

```

1 param name: T
2 def name$default: T = rhs

```

At the place where an optional context parameter is used, the default function is called in the absence of a binding for the parameter.

Optional context parameters are more powerful than optional parameters supported in languages such as Python and Scala. This is because it is possible to pass optional context parameters deep into the call stack without incurring syntactic overhead (Section 4).

2.4 Controlling Context Parameters

Context parameters give programmers a super power. As is widely known, any unchecked power is a danger. We need a mechanism to control the usage of context parameters.

A concrete example is command line arguments: the language could expose them as context parameters to simplify `main`, while an architect may still disallow submodules from relying on such brittle assumptions directly.

We provide the construct `allow` to control the usage of context parameters:

```

1 param maxResultCount: Int // maximum result count
2 param connection: Connection // database connection
3 param args: Array[String] // command-line args
4
5 def search(keyword: String) = ...
6
7 def process = allow connection in search(keyword) with maxResultCount = 200

```

² It seems possible to relax the constraint to a little to allow usage of optional context parameters. However, that opens the door for potential cycles and semantic surprises. We choose to be more conservative here.

The usage of `with` and `allow` in the code above ensures that the function `search` can use only the context parameters `maxResultCount` and `connection`. Only the context parameters listed after `allow` may be validly used to evaluate the expression before it. The binding for context parameter `args` from the outer calling context cannot be used during the execution of the search.

The special syntax `allow none` disallows usage of any context parameters from the enclosing calling context, as the following example demonstrates:

```
1 def test = allow none in factorial(10)
```

As a consequence, the function `factorial` may not directly or indirectly use any context parameters from outside.

Given the following program

```
1 param newLine: String
2
3 def foo() =
4   print
5     "Hello, world!" + newLine
6
7 def bar() = foo()
8
9 def main = allow stdout in bar
```

The compiler will produce the following error:

```
----- Error at allow.jo:9:12 -----
| def main = allow stdout in bar
|                               ~~~
|                               Parameter not allowed: newLine

The following is the trace that leads to the problem:
| def main = allow stdout in bar [ allow.jo:9:12 ]
|                               ~~~
| def bar() = foo [ allow.jo:7:13 ]
|               ~~~
|               "Hello, world!" + newLine [ allow.jo:5:23 ]
|               ~~~~~
```

2.5 First-Class Functions

First-class functions are now ubiquitous in mainstream languages. Enabling automatic capture of context parameters is important for usability, similar to function parameters. Unlike function parameters, where the captured parameters are local, context parameters may be captured from a deep call chain:

```
1 param a: Int
2 param b: Int
3
4 def makeFun(x: Int) = (n: Int) => n + a * bar(x)
5
6 def bar(x: Int): Int = x * b
7
8 def main =
```



```

358 9   val f = makeFun(3) with a = 5, b = 10
359 10   f(20)
360

```

In the code above, the closure created in `makeFun` not only captures the context parameter `a`, but also indirectly captures `b`. This way, when we call the created function `f`, there is no need to supply any context parameters.

We intentionally select the design that *by default*, a closure captures all the context parameters that it depends on at its creation site. This design is based on two considerations: (1) it simplifies the usage of context parameters; (2) it is what programmers usually want unless specified otherwise.

However, sometimes the programmer might want to receive the value of a context parameter from the call site of a closure, instead of the creation site. A programmer can selectively tune the capture behavior by declaring call-site dependencies as follows:

```

371 1   param pageWidth: Int = 78
372 2
373 3
374 4   type Doc = String
375 5   def pretty(doc: Doc): String receives pageWidth = ...
376 6
377 7   type Printer = Doc => String receives pageWidth // use type alias for
378 8       readability
379 9   def createPrinter(): Printer receives none =
380 10       (doc: Doc) => pretty(doc)
381 11
382 12   def main =
383 13       val printer = createPrinter()
384 14       printer("hello, world!") with pageWidth = 100 // printer uses pageWidth = 100
385

```

In the above, the keyword `receives` at line 4 specifies that the *2nd-class* function `pretty` takes the context parameter `pageWidth` — the specification will trigger the compiler check to ensure the actual dependencies conform to the specification.

We also use the keyword `receives` in the type alias `Printer` at line 6. There it specifies that a *first-class function* of this type take the context parameter `pageWidth` from its call site instead of capturing it from its creation site. Consequently, the lambda inside `createPrinter` at line 8 will not require `pageWidth` at the creation site, which is made explicit with the specification `receives none` at line 7.

The next section will show how to implement selective deep capture in first-class functions based on a type system (Section 3).

3 The Deep Capture Problem

As we have seen in the previous section, first-class functions transitively and selectively capture all of their dependencies on context parameters. But how can we know the set of transitive dependencies statically at compile time?

In this section, we present a type system that can effectively compute the deep capture set of a closure based on a small calculus λ^{CP} . The type system also guarantees the safe usage of context parameters, i.e., a context parameter is always bound when it is used. Meanwhile, it enables static control of the usage of context parameters using `allow`-expressions. We provide a soundness proof in Appendix A. To keep the calculus small and to the essence, we do not model optional context parameters.

Terms	Program	Dynamic context
$t ::= x$	$\mathbb{P} ::= (\text{param } \bar{p} : \bar{T}, t)$	$\rho ::= \emptyset \mid \rho, x \mapsto v$
$\mid \lambda(x:T)\{\bar{p}\}.t$	Types	$\psi ::= \emptyset \mid \psi, p \mapsto v$
$\mid t u$	$T ::= B \mid T\{\bar{p}\} \rightarrow T$	Typing Context
$\mid p$	Values	$\Gamma ::= \emptyset \mid \Gamma, p : T \mid \Gamma, x : T$
$\mid t \text{ allow } \bar{p}$	$v ::= \langle \lambda x:T.t, \rho, \psi \rangle$	
$\mid t \text{ with } p = u$		

■ Figure 1 Syntax of λ^{CP} .

3.1 The Language

The syntax of λ^{CP} is given in Figure 1. It is an extension of the simply-typed lambda calculus. A program $\mathbb{P}$ is defined as a set of context-parameter declarations, followed by a term.

A lambda abstraction $\lambda(x:T)\{\bar{p}\}.t$ is explicitly annotated with the set of context parameters $\bar{p}$ that *should not be captured* at definition site, but instead provided at call site. The value of a context parameter can be bound using the **with** construct. The set of context parameters available to a nested execution context can be controlled with the **allow** construct.

Types are also given in Figure 1. Besides base types B , the arrow type $T\{\bar{p}\} \rightarrow S$ is extended to state the set of context parameters $\bar{p}$ that are expected at call site. Following common convention, we assume the names are unique to simplify the presentation.

3.2 Semantics

The big-step semantics of the language are given in Figure 2. An evaluation relation takes the form $\llbracket t \rrbracket(\rho, \psi) \longrightarrow v$, where the term t evaluates to the value v given a normal parameter environment ρ and a context parameter environment ψ . Values v are closures that contain a function and its associated pair of environments. Most rules are straightforward; we explain only the interesting ones.

Evaluating a lambda abstraction (Rule E-Abs) introduces a closure that captures the whole normal parameter environment ρ , but not the whole set of context parameters: the excluded parameters $\bar{p}$ are removed in the closure.

When evaluating an application (Rule E-App), the current environments are merged with the ones captured in the closure, which might *shadow* bindings from the call-site in favor of the captured ones.

The rule E-Allow restricts the set of context parameters for the subterm. For efficiency, this restriction, however, is not needed in an actual implementation, as the type system presented below guarantees that only the allowed context parameters can be used in evaluating the subterm.

Evaluating a **with** expression (Rule E-With) creates a binding of p to the value v in the dynamic context for evaluating the subterm t .

3.3 Type System

The language is checked by the typing rules in Figure 3, which are in the style of a type-and-effect system [7].

In the typing judgment $\Gamma \vdash t : T \mid \Psi$, the term t and the typing environment Γ are given as input, and the resulting type T and set of *used* context parameters Ψ are computed as output. Intuitively, the dynamic environment of bound context parameters ψ should cover the set Ψ for the evaluation to succeed.

$$\begin{array}{c}
\text{E-VAR} \\
\llbracket x \rrbracket(\rho, \psi) \longrightarrow \rho(x)
\end{array}
\quad
\begin{array}{c}
\text{E-CVAR} \\
\llbracket p \rrbracket(\rho, \psi) \longrightarrow \psi(p)
\end{array}
\quad
\begin{array}{c}
\text{E-ABS} \\
\frac{v = \langle \lambda x:T.t, \rho, (\psi \setminus \bar{p}) \rangle}{\llbracket \lambda(x:T)\{\bar{p}\}.t \rrbracket(\rho, \psi) \longrightarrow v}
\end{array}$$

$$\begin{array}{c}
\text{E-APP} \\
\frac{\llbracket t \rrbracket(\rho, \psi) \longrightarrow \langle \lambda x:T.t', \rho', \psi' \rangle \quad \llbracket u \rrbracket(\rho, \psi) \longrightarrow v \quad \llbracket t' \rrbracket(\rho' \cup x \mapsto v, \psi \cup \psi') \longrightarrow v'}{\llbracket t u \rrbracket(\rho, \psi) \longrightarrow v'}
\end{array}
\quad
\begin{array}{c}
\text{E-ALLOW} \\
\frac{\psi' = \psi|_{\bar{p}} \quad \llbracket t \rrbracket(\rho, \psi') \longrightarrow v}{\llbracket t \text{ allow } \bar{p} \rrbracket(\rho, \psi) \longrightarrow v}
\end{array}$$

$$\begin{array}{c}
\text{E-WITH} \\
\frac{\llbracket u \rrbracket(\rho, \psi) \longrightarrow v \quad \llbracket t \rrbracket(\rho, \psi[p \mapsto v]) \longrightarrow v'}{\llbracket t \text{ with } p = u \rrbracket(\rho, \psi) \longrightarrow v'}
\end{array}
\quad
\begin{array}{c}
\text{E-PROG} \\
\frac{\llbracket t \rrbracket(\emptyset, \emptyset) \longrightarrow v}{\llbracket (\text{param } p:T, t) \rrbracket(\emptyset, \emptyset) \longrightarrow v}
\end{array}$$

■ **Figure 2** Big-step evaluation rules of λ^{CP} .

The typing judgment relies on a sub-judgment of *type well-formedness*, written $\Gamma \vdash T$, which simply ensures that all context parameter names mentioned in T are bound in Γ . We omit the rules for the sake of presentation. Most rules are straightforward; we explain only the interesting ones below.

For a lambda abstraction (Rule T-Abs), the set of context parameters needed is obtained by removing the call-site-provided ones $\bar{p}$ from the set needed by the body of the function Ψ . For a function application (Rule T-App), the sets of context parameters are accumulated.

The rule T-Allow enforces the programmer-defined policy: only the allowed context parameters may be used in evaluating the subterm. In the rule for context parameter binding (Rule T-With), the bound parameter p is removed from the set of needed parameters, which is similar to *effect masking* in type-and-effect systems.

Finally, the non-syntax-directed rule (Rule T-Sub) allows to artificially extend the set of required context parameters. This rule is motivated to compensate for the absence of subtyping for types, which is a standard trick in the literature [15].

Typing a program (Rule T-Prog) amounts to typing the body of the program in the typing environment of context parameters. The body of the program should not depend on any context parameters provided from outside — it should be self-sufficient.

The top-level 2nd-class functions in our language can be modeled by lambdas. However, as they never capture context parameters, we can infer the list of non-captured context parameters:

$$\frac{\Gamma \vdash T \quad \Gamma, x : T \vdash t : S ! \Psi}{\Gamma \vdash \lambda(x:T)\{*\}.t : T\{\Psi\} \rightarrow S ! \emptyset}$$

In the above, we use the special syntax $*$ to indicate that all context parameters used inside the lambda come from call sites. To keep the calculus simple and to the essence, we treat the new lambda form as a syntactic sugar.

We prove that a well-typed program always evaluates to a value (Appendix A). Formally,

► **Theorem 1 (Soundness).** *Given a well-typed program, where we have $\Vdash (\text{param } p_i:T_i, t) : S$, evaluating the program cannot get stuck, i.e., $\llbracket t \rrbracket(\emptyset, \emptyset) \longrightarrow v$ for some value v .*

The type system ensures the safe usage of context parameters at runtime, i.e., a used context parameter is guaranteed to be provided in an outer context. Moreover, it provides a

$$\begin{array}{c}
\text{T-VAR} \quad \frac{x : T \in \Gamma}{\Gamma \vdash x : T ! \emptyset} \quad \text{T-CVAR} \quad \frac{p : T \in \Gamma}{\Gamma \vdash p : T ! \{p\}} \quad \text{T-ABS} \quad \frac{\Gamma \vdash T \quad \bar{p} \in \Gamma \quad (\Gamma, x : T) \vdash t : S ! \Psi}{\Gamma \vdash \lambda(x:T)\{\bar{p}\}.t : T\{\bar{p}\} \rightarrow S ! (\Psi \setminus \bar{p})} \\
\\
\text{T-APP} \quad \frac{\Gamma \vdash t : S\{\bar{p}\} \rightarrow T ! \Psi \quad \Gamma \vdash u : S ! \Psi'}{\Gamma \vdash t u : T ! \Psi \cup \Psi' \cup \bar{p}} \quad \text{T-ALLOW} \quad \frac{\Gamma \vdash t : T ! \Psi \quad \bar{p} \in \Gamma \quad \Psi \subseteq \bar{p}}{\Gamma \vdash t \text{ allow } \bar{p} : T ! \bar{p}} \\
\\
\text{T-WITH} \quad \frac{p : T \in \Gamma \quad \Gamma \vdash u : T ! \Psi \quad \Gamma \vdash t : S ! \Psi'}{\Gamma \vdash t \text{ with } p = u : S ! \Psi \cup (\Psi' \setminus p)} \quad \text{T-SUB} \quad \frac{\Gamma \vdash t : T ! \Psi \quad p \in \Gamma}{\Gamma \vdash t : T ! \Psi \cup \{p\}} \\
\\
\text{T-PROG} \quad \frac{\forall i < n, p_0 : T_0, \dots, p_{i-1} : T_{i-1} \vdash T_i \quad \Gamma = \overline{p_i : T_i} \quad \Gamma \vdash t : S ! \emptyset}{\Vdash (\text{param } p_i : \overline{T_i}, t) : S}
\end{array}$$

■ **Figure 3** Typing rules for terms ($\vdash$) and programs ($\Vdash$).

mechanism for the upstream of a call chain to statically control what context parameters may be passed downstream to submodules — only statically allowed context parameters are guaranteed to be provided thus may be used in downstream calls.

The type system also affords a solution to the *deep capture problem* in the presence of first-class functions.

3.4 Deep Capture

After type checking, transitive dependencies of context parameters in lambdas are computed by the rule T-Abs. The compiler can then take advantage of the information to create closures that only capture those context parameters (excluding the ones specified to be provided from call sites). This improves the performance of the implementation compared to blindly capturing all context parameters, as it is done in the semantic rule E-Abs.

Conceptually, the handling of selective capture of context parameters needs to come before closure conversion. A way of achieving selective capture is to create aliases of the values of the captured context parameters at the outside of the lambda and insert code to bind them inside the lambda:

```

1 def foo() =
2   val v1 = param1
3   val v2 = param2
4   // ...
5   (x: Int) =>
6     ...
7   with param1 = v1, param2 = v2, ...

```

The transform introduces local variables `v1`, `v2` and so on that are captured in the closure. That will be handled by closure conversion.

The type system guarantees that in evaluating the expression $t \text{ allow } \bar{p}$, only the allowed context parameters will be used. Consequently, there is no need to actually narrow the context parameters in the implementation as it is done in E-Allow.

4 Case Studies

In this section, we show several use cases of context parameters. It turns out that several different problems in real-world programming are instances of the problem of parameterization. such as contextual abstraction, scalable optional parameters, dependency injection, and software evolution. Consequently, context parameters lend themselves as a solution to these problems.

4.1 Contextual Abstraction

In the following, we present an interpreter for untyped lambda calculus with call-by-value semantics:³

```

1  union Expr = Abs(x: String, body: Expr) | Var(x: String) | App(lhs: Expr, arg:
2  Expr)
3  union Env = Empty | Cons(k: String, v: Value, outer: Env)
4  class Value(lambda: Abs, env: Env)
5  param env: Env = Empty // environment for variables
6
7  def find(x: String): Option[Value] receives env = // use only 'env'
8  match env
9  case Empty => None
10
11  case Cons k v outer =>
12    if x == k then Some(v)
13    else find x with env = outer
14
15  def eval(expr: Expr): Value = // inferred usage of 'env'
16  match expr
17  case Var x =>
18    match find x
19    case None => abort ("Unfound variable: " + x)
20    case Some v => v
21  end
22
23  case abs: Abs =>
24    Value abs env
25
26  case App abs arg =>
27    val closure = eval abs
28    val argValue = eval arg
29    val x = closure.lambda.x
30    val body = closure.lambda.body
31    val env2: Env = Cons(x, argValue, closure.env)
32    eval body with env = env2 // binding 'env'

```

One piece of contextual information that is needed to evaluate an expression is the set of value bindings for names. By abstracting value bindings as a context parameter, there is no need to pass them as an additional parameter in function calls in the interpreter, which avoids syntactic overhead.

³ Union definitions is the way to define algebraic data types in our language.

More generally, context parameters can be used as a convenient mechanism for contextual abstraction. For example, they could propagate the rendering context for a graphics program, the typing context for a type checker, etc. The common trait of these use cases is that the context information remains the same in most of the nested calls. As a result, the additional parameter(s) usually get passed around unchanged, and thus become boilerplate.

4.2 Scalable Optional Parameters

Many modern languages support optional parameters, e.g., Scala, OCaml, Python. One problem with the design of optional parameters is that they do not scale. This is better illustrated with a simple example in Scala:

```
1 def foo(n: Int, flip: Boolean = false): Int = if flip then -n else n
2 def bar(n: Int, double: Boolean = false): Int = if double then n * 2 else n
3 def baz(n: Int): Int = foo(n) + bar(n)
```

In the example above, both the functions `foo` and `bar` take optional parameters. When we encapsulate the two functions in `baz`, we would like the optional parameters of `foo` and `bar` to become the optional parameters of `baz`. In existing languages, programmers have to manually re-declare the optional parameters for the function `baz`. When there are more than 10 optional parameters or the calling chain is longer than 3, this incurs overwhelming boilerplate and thus discourages programmers from creating libraries that are deeply parameterizable.

With context parameters, the optional parameters are automatically propagated:

```
1 param flip: Bool = false // flip the sign
2 param double: Bool = false // double the value
3
4 def foo(n: Int): Int = if flip then 0 - n else n
5 def bar(n: Int): Int = if double then n * 2 else n
6
7 // The optional context param 'flip' and 'double' are automatically active.
8 def baz(n: Int): Int = foo(n) + bar(n)
```

4.3 Dependency Injection

In the Java world, *inversion of control* and *dependency injection* are important topics. In a popular blog post,⁴ Martin Fowler explains that the central problem is the following: in the following Java code example, how can one wire an instance of `MovieFinder` to be used inside the method `MovieLister.moviesDirectedBy`?

```
1 public interface MovieFinder { List findAll(); }
2
3 class MovieLister...
4     public Movie[] moviesDirectedBy(String arg) {
5         List allMovies = finder.findAll();           // use of finder
6         for (Iterator it = allMovies.iterator(); it.hasNext();) {
7             Movie movie = (Movie) it.next();
8             if (!movie.getDirector().equals(arg)) it.remove();
9         }
10    }
```

⁴ <https://martinfowler.com/articles/injection.html>

```

592   9      }
593  10      return (Movie[]) allMovies.toArray(new Movie[allMovies.size()]);
594  11  }
595

```

More generally, the core problem in dependency injection is how to provide a service to a submodule without depending on any implementation details of the service. If we regard a service as an additional parameter, the problem becomes the following: *How to provide a parameter of an abstract service type to a submodule without incurring too much syntactic overhead?*

The solutions mentioned in the blog post vary in terms of syntactic verbosity. All solutions resort to some type-based tags to do the wiring, which are verbose and unsafe — there is no guarantee that the required service is present.

In our language, we can solve the problem easily with context parameters:

```

605  1  class Movie(name: String, director: String, year: Int)
606  2  interface MovieFinder
607  3      def findAll(): List[Movie]
608  4
609  5  param finder: MovieFinder
610  6
611  7  def moviesDirectedBy(String arg): List[Movie] =
612  8      val allMovies = finder.findAll()
613  9      def filter(movies: List[Movie]): List[Movie] =
614 10          match movies
615 11              case [] => []
616 12              case [m, ..rest] =>
617 13                  if m.director == arg then [m, ..(filter rest)]
618 14                  else filter rest
619 15          end
620 16      filter allMovies
621
622

```

Most popular Java frameworks support circular dependencies,⁵ even though they are widely recognized as problematic due to obscure semantics and subtle initialization. With context parameters, circular dependencies can be made safe:

```

626  1  interface ServiceFoo
627  2      def foo(): Unit receives bar
628  3
629  4  interface ServiceBar
630  5      def bar(): Unit receives foo
631  6
632  7  param foo: ServiceFoo
633  8  param bar: ServiceBar
634
635

```

In the above, the service `foo` depends on `bar` and vice versa. Context parameters enable both static control of dependencies as well safe initialization: The static type system ensures that no context parameters may be used without being bound.

⁵ For example, the popular Java framework Quarkus supports circular dependencies: <https://github.com/quarkusio/quarkus/issues/2990>

639 4.4 Software Evolution

640 Software evolution involves adapting the existing behavior of a program to new environments.
 641 That change is usually implemented by adding new parameters to existing code such that it
 642 can change its behavior according to the additional parameter. Context parameters facilitate
 643 such evolution by minimizing changes to the existing code base.

644 To make things more concrete, let us consider the example of a program that pretty
 645 prints the content of a CSV file. Suppose that initially the program only supports UNIX-like
 646 files, where a line ends with a single new line character `'\n'`.

647 In a traditional programming language, it is reasonable that an experienced programmer
 648 encodes the line ending in a global constant and refers to the global constant in the program:

```
649  
650 1 val lineEnding: String = "\n"  
651
```

652 After release of the program, there arrives the requirement to support Windows-like
 653 files, where lines end with both a carriage return and a new line character `"\r\n"`. Now,
 654 the programmer has two choices: (1) make `lineEnding` a global mutable variable, which
 655 is a code smell but it minimizes the changes; or (2) pass `lineEnding` from top level of the
 656 program to all the places it is used, which is safer but incurs invasive changes. Neither of the
 657 choices is ideal.

658 However, with context parameters, the programmer can still define `lineEnding` as a
 659 constant in the initial version:

```
660  
661 1 def lineEnding: String = "\n"  
662
```

663 Later, when the requirement changes, it can be easily addressed by changing the keyword
 664 `fun` to `param`:

```
665  
666 1 param lineEnding: String = "\n"  
667
```

668 At the top level, the parameter can be derived from actual file content based on heuristics
 669 or supplied explicitly by the user.

670 The most important thing is that by introducing the additional parameter, no signatures
 671 of existing functions need to be changed. The top-level parameter passes from the program
 672 entry to the places where it is needed implicitly.

673 5 Towards Practical Object-Capability Languages

674 The object-capability model is the primary approach in designing secure systems [9]. Based
 675 on the principle of *connection begets connection*, the object-capability model enables reasoning
 676 about security. Despite its advantages, there has not been wide adoption of the model in
 677 practical programming languages.

678 5.1 The Curse of Ambient Authorities

679 If the object-capability model is so simple, why do most popular languages not support
 680 designing and reasoning about security, e.g., safe execution of untrusted code generated by
 681 large-language models? Reasoning about security is undermined by *ambient authorities*.
 682 The concern for security gives way to convenience of usage in popular languages. There are
 683 several sources of ambient authorities in popular languages:

- 684 ■ Global state introduced by global variables

- 685 ■ Native platform calls exposed in the standard library and 3rd-party libraries
- 686 ■ Reflective language features, e.g., Java reflection and runtime class method update in Ruby
- 687 ■ Unexpected sharing via control effects

689 Consequently, in accordance with the principle of least privileges, the recipe for making a
 690 secure language is to remove all ambient authorities, as exemplified in the design of Joe-E by
 691 Mettler et al. [8]. The benefits for security are immediate: easier reasoning about security
 692 and security auditing.

693 However, we believe one essential usability problem is overlooked by past research on
 694 object-capability systems: *how to pass the authorities in the program in a convenient way?*
 695 If there are 10 different authorities needed by a submodule, passing them along the call
 696 chain would discourage programmers. This is essentially the problem of parameterization
 697 (Section 1).

698 Context parameters provide the mechanism for static control of ambient authorities
 699 without incurring syntactic overhead in passing the authorities in the program. Below, we
 700 show how to design a capability-based secure language on top of context parameters.

701 5.2 The Design of A Secure Language

702 With context parameters, we can remove the feature of global variables from the language.
 703 We also restrict the language to not have control effects (such as structural exceptions) nor
 704 reflective language features. The remaining question is how to design the API for side effects
 705 in the language.

706 We will use file I/O as an example, given that on Unix-like systems, it is easy to generalize
 707 to other kinds of side effects. The file API of our language is designed based on context
 708 parameters: ⁶

```

709 1 param open: String => File      // Open a file or create one if absent
710 2 param stdin: Unit => String
711 3 param stdout: String => Unit
712 4
713 5 interface File
714 6   def close(): Unit
715 7   def seek(pos: Int): Unit
716 8   def hasMore(): Bool
717 9
718 10  def readLine(): String
719 11  def write(content: String): Unit
720
721 
```

722 The `File` interface contains methods that enable reading from and writing to files. A file
 723 is created by using the context parameter `open`. The interface for side effects is completely
 724 parametric at the language level. The context parameters `stdout`, `stdin` and `open` are
 725 provided to the main program by the language runtime.

726 The root capabilities such as `open` are provided by default to the `main` function:

```

727 1 def main receives IO.open, IO.stdout, IO.stdin = ...
728
729 
```

⁶ For the sake of illustration, we make the API simple. More practical API design needs to include options and error handling.

We may think of the context parameters provided by the language runtime as root capabilities in the object-capability model [9]. In disciplined system design, it makes sense to only provide the minimal and necessary capabilities to submodules by providing fine-grained capabilities derived from the root capabilities.

For example, if a submodule counts the number of lines in a file, we can define a fine-grained capability `readLine` as a context parameter as follows:

```

736 1 param readLine: () => Option[String]
737 2
738 3
739 4 def lineCount() receives readLine =
740 5   def recur(acc: Int) =
741 6     readLine match
742 7       case Some(_) => recur (acc + 1)
743 8       case None   => acc
744 9   end
745 10  recur 0
746 11
747 12 def main =
748 13   val filePath = "data.txt"
749 14   val file = open filePath
750 15   val readLineFun = () =>
751 16     if file.hasMore then Some(file.readLine())
752 17     else None
753 18
754 19 allow none in lineCount() with readLine = readLineFun
755

```

In the code above, we use `allow none` to make sure that the execution of the function `lineCount` cannot use any other capabilities except those provided by `readLine`. That endows software architects with a powerful tool to enforce fine-grained control of side effects in software modules.

Meanwhile, the capability requirement of the function `lineCount` is parametric. We can easily test the function `lineCount` as follows:

```

762 1 class MockFile
763 2   var line: Int = 0
764 3   def read(): Option[String] =
765 4     if line == 0 then
766 5       line = line + 1
767 6       Some("abc")
768 7     else
769 8       None
770 9
771 10
772 11 def test() =
773 12   val mockFile = new MockFile
774 13   val readLineFun = () => mockFile.read()
775 14   allow none in lineCount() with readLine = readLineFun
776

```

6 Related Work

Context parameters bear similarity to dynamic scoping in Lisp-family languages. Dynamic scoping is best exemplified by *special variables* in Common Lisp [13]:

```

780 1 (defun foo (x)
781

```

```

782 2 (let ((factor 10))
783 3   (declare (special factor))
784 4   (bar x)))
785 5
786 6 (defun bar (n)
787 7   (declare (special factor))
788 8   (* factor n))
789 9
790 10 (format t "~D" (foo 10)) ; prints 100
791

```

In the example above, the function `foo` passes the extra parameter `factor`, which is called a special variable, to the function `bar` implicitly. This is a lot of power, but unfortunately the common consensus in the programming language community is that *dynamic scoping is bad*.

Lewis et al. [6] propose *implicit parameters* to tame dynamic scoping with typing discipline. In their work, a function using implicit parameters takes a different type signature:

```

798 1 pretty :: (?width :: Int) => Doc -> String
799 2 pretty doc =
800 3   ... if i >= ?width then ...
801 4
802 5 prettyTwice :: (?width :: Int) => Doc -> String
803 6 prettyTwice d = pretty d ++ pretty d
804 7
805 8 prettyTwice d with ?width = 78
806
807

```

In the code above, the function `pretty` takes the implicit parameter `?width`, which is reflected in its type. The requirement for the implicit parameter propagates to the function `prettyTwice`: its type specifies that it takes the implicit parameter `?width` as well. The user may supply the implicit parameter using the keyword `with`. The dynamic binding is only effective for evaluating the expression on the left-hand side of the `with` expression.

Implicit parameters constitute a big step forward — they keep the essence of special variables as a simple and safe language feature while avoiding the complexities and pitfalls.

Steele [13] provides a good understanding of various parameters in terms of *scope* and *extent* (Chapter 3, Scope and Extent):

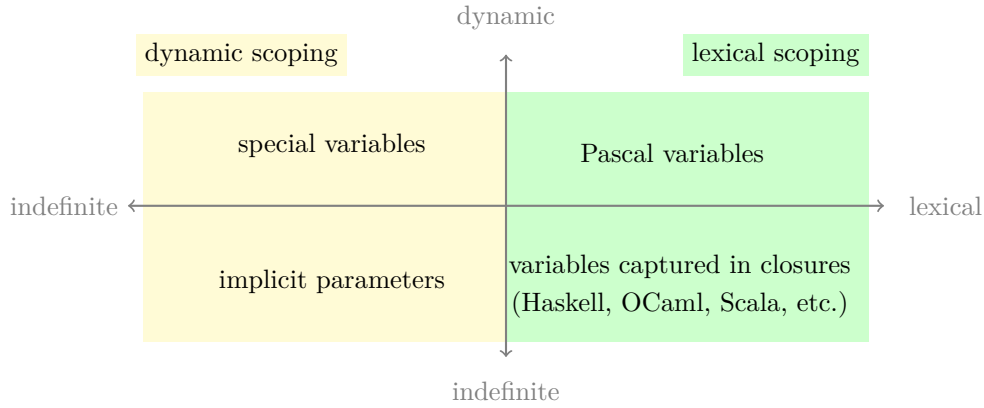
- **Scope:** The textual region of the program where a name can be used.
- **Extent:** The interval of time at run time during which a name can be used.

The two dimensions are illustrated in Figure 4. The horizontal axis represents the dimension of scope – the life space of a name in the textual program. In terms of scope, a name can be either

- *lexical*, in that it may be only used within a well-defined textual region, as is the case for variables in almost all mainstream languages; or
- *indefinite*, in that there is no restriction in terms of textual region where a name can be used, as is the case for implicit parameters [6] and special variables in Common Lisp.

The vertical axis represents extent – the life time of a name at execution time. In terms of extent, a name can be either dynamic or indefinite:

- *Dynamic* extent means that there is a well-defined interval at run time during which the name can be validly used. Such intervals usually follow a stack discipline. This is the case for variables in Pascal and special variables in Common Lisp.



■ **Figure 4** Scope and extent. Scope refers to the textual region of the program where a name can be used, represented by the horizontal line. Extent refers to the time interval during which a name can be used, represented by the vertical line.

■ *Indefinite* extent means that there is no restriction on the life time of the name. This is the case for functional programming languages where a name may be captured in closures, so it has an indefinite life time until the garbage collector recycles the closure when it is no longer reachable.

Note that in Figure 4, we classify implicit parameters in the category of indefinite extent. This is because the following code will return 80

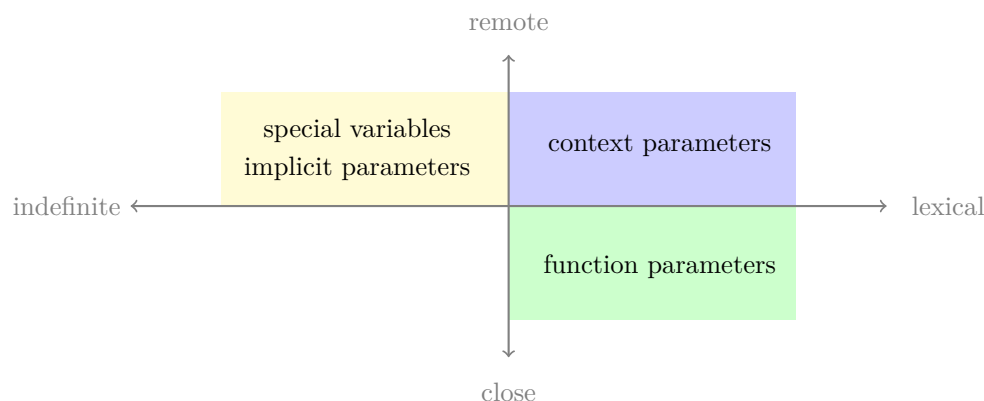
```
1 ((\x. x + ?width) with ?width = 78) 2)
```

while it throws an error for special variables:

```
1 (funcall
2   (let ((width 78))
3     (declare (special width))
4
5     (lambda (x) (+ x width)))
6   2)
```

However, one dimension that is not captured in the categorization by Steele [13] is that of *binding distance*. By binding distance, we mean the difference of the calling stack depth between the binding time of a parameter and its usage time. We illustrate the design space in Figure 5. For function parameters, the stack distance between the binding of a parameter and its usage is short, which we call *close binding*. In contrast, implicit parameters [6] and special parameters in Common Lisp take advantage of remote binding — a binding can be effective in a deep stack context. We believe that remote binding is the essential appealing aspect of what is commonly called dynamic scoping, rather than the non-lexical aspect. Context parameters embrace both remote binding and lexical scope.

In the work of Lewis et al. [6], first-class functions may not take implicit parameters. In contrast, in our work, first-class functions can take context parameters. Implicit parameters are essentially typed tags which can be used without explicit declaration. Unlike implicit parameters, context parameters need to be explicitly declared. Consequently, it is possible to organize them in hierarchical namespaces and have a single place to document contracts and invariants associated with a context parameter. Moreover, their work lacks a design for optional parameters, which are essential if the feature is to be an alternative to global



■ **Figure 5** Scope and binding distance of parameters.

865 variables.

866 Context parameters also bear similarity to *scoped values* in Java,⁷ which is also inspired
 867 by dynamically scoped variables in Lisp dialects. However, scope values in Java do not
 868 guarantee the safety of usage, nor can a lambda transitively capture all scoped values — the
 869 so-called *deep capture* problem we solved via a static type system.

870 Scala popularized implicits in programming, which are widely used for contextual ab-
 871 straction [10]. Scala-style implicits are based on a fundamentally different design choice:
 872 *type-based inference*. In contrast, context parameters are *name-based*. We believe the two
 873 design choices are complementary.

874 For some use cases, the name-based approach seems to be simpler, as evidenced by the
 875 following idiomatic Scala code:

```
876
877 1 def ctx(using ctx: Context): Context = ctx
878
```

879 The type-based approach mandatorily contaminates the signature of all functions in the
 880 call chain, while it is optional for the name-based approach.

881 However, for use cases such as type class evidences, type-based inference is a better fit,
 882 as the following example demonstrates:

```
883
884 1 def show[T](v: T)(using ev: Show[T]): String = ev.show(v)
885
```

886 In fact, context parameters cannot be used for the scenario above, because they are global
 887 and may not be parameterized by a type parameter — at least for the design presented in
 888 the current work.

889 The recent introduction of algebraic effects [11, 12, 2] makes it easy to define custom
 890 effects and handlers, which can also be used to achieve effect parameterization. In terms of
 891 expressiveness, algebraic effects are more powerful. However, context parameters are much
 892 simpler for programmers and language designers. Meanwhile, some use cases of context
 893 parameters, such as optional parameters and contextual abstraction, do not fall naturally in
 894 the domain of algebraic effects.

895 Interestingly, we observe reluctance among designers of some real-world programming
 896 languages to add control-flow effects. For example, Rust does not support `try/catch`. We
 897 hope that context parameters may qualify as a design choice in these scenarios thanks to
 898 their simplicity.

⁷ <https://openjdk.org/jeps/506>

7 Conclusion

In the paper, we highlighted the problem of parameterization and proposed context parameters as an alternative design to global variables. Context parameters enable programmers to achieve higher parameterization of software modules with a low cost. Consequently, we expect context parameters to improve testability and reusability of software modules.

We see that context parameters have the potential to address several real-world programming problems, such as contextual abstraction, scalability of optional parameters, dependency injection, software evolution, and secure programming.

To summarize, we believe context parameters will be a valuable addition to a language designer's toolbox and will bring a new vision for programming and language design.

References

- 1 Anindya Banerjee, David A Naumann, and Stan Rosenberg. Regional logic for local reasoning about global invariants. In *European Conference on Object-Oriented Programming*, pages 387–411. Springer, 2008.
- 2 Jonathan Immanuel Brachthäuser, Philipp Schuster, Edward Lee, and Aleksander Boruch-Gruszecki. Effects, capabilities, and boxes: from scope-based reasoning to type-based reasoning and back. *Proceedings of the ACM on Programming Languages*, 6(OOPSLA1):1–30, 2022.
- 3 Charles Antony Richard Hoare. *The emperor's old clothes*. Association for Computing Machinery, New York, NY, USA, 1980. URL: <https://doi.org/10.1145/1283920.1283936>.
- 4 Butler W Lampson. A note on the confinement problem. *Communications of the ACM*, 16(10):613–615, 1973.
- 5 K Rustan M Leino and Peter Müller. Modular verification of static class invariants. In *International Symposium on Formal Methods*, pages 26–42. Springer, 2005.
- 6 Jeffrey R Lewis, John Launchbury, Erik Meijer, and Mark B Shields. Implicit parameters: Dynamic scoping with static types. In *Proceedings of the 27th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pages 108–118, 2000.
- 7 John M. Lucassen and David K. Gifford. Polymorphic effect systems. In Jeanne Ferrante and P. Mager, editors, *Conference Record of the Fifteenth Annual ACM Symposium on Principles of Programming Languages, San Diego, California, USA, January 10-13, 1988*, pages 47–57. ACM Press, 1988. doi:10.1145/73560.73564.
- 8 Adrian Mettler, David A Wagner, and Tyler Close. Joe-e: A security-oriented subset of java. In *NDSS*, volume 10, pages 357–374, 2010.
- 9 Mark S. Miller, Chip Morningstar, and Bill Frantz. Capability-based financial instruments. In Yair Frankel, editor, *Financial Cryptography, 4th International Conference, FC 2000 Anguilla, British West Indies, February 20-24, 2000, Proceedings*, volume 1962 of *Lecture Notes in Computer Science*, pages 349–378. Springer, 2000. doi:10.1007/3-540-45472-1_24.
- 10 Martin Odersky, Olivier Blanvillain, Fengyun Liu, Aggelos Biboudis, Heather Miller, and Sandro Stucki. Simplicitly: Foundations and applications of implicit function types. *Proceedings of the ACM on Programming Languages*, 2(POPL):1–29, 2017.
- 11 Gordon Plotkin and Matija Pretnar. Handlers of algebraic effects. In *European Symposium on Programming*, pages 80–94. Springer, 2009.
- 12 Gordon D Plotkin and Matija Pretnar. Handling algebraic effects. *Logical methods in computer science*, 9, 2013.
- 13 Guy Steele. *Common LISP: the language*. Elsevier, 1990.
- 14 Michael J Steindorfer and Jurgen J Vinju. Optimizing hash-array mapped tries for fast and lean immutable jvm collections. In *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications*, pages 783–800, 2015.

- 947 15 Philip Wadler and Peter Thiemann. The marriage of effects and monads. *ACM Transactions*
 948 *on Computational Logic (TOCL)*, 4(1):1–32, 2003.
- 949 16 W. Wulf and Mary Shaw. Global variable considered harmful. *SIGPLAN Not.*, 8(2):28–34,
 950 February 1973. doi:10.1145/953353.953355.

951 A Soundness

952 This section presents a soundness proof for the λ^{CP} calculus. The main result is that
 953 well-typed programs do not get stuck due to accessing undefined context parameters.

954 A.1 Semantic Interpretation

955 We first define what it means for runtime values to inhabit types, and for runtime environments
 956 to satisfy typing contexts.

957 ► **Definition 2** (Value Typing). *We define the semantic interpretation of types $\mathcal{V}[[T]]$ by*
 958 *induction on the structure of T :*

$$\begin{aligned}
 959 \quad \mathcal{V}[[B]] &= \text{the set of base values of type } B \\
 960 \quad \mathcal{V}[[T\{\bar{p}\} \rightarrow S]] &= \{ \langle \lambda x:T.t, \rho, \psi \rangle \mid \forall v \in \mathcal{V}[[T]], \forall \psi' \text{ such that } \psi' \models_p \bar{p}, \\
 961 \quad &\quad \llbracket t \rrbracket((\rho \cup \{x \mapsto v\}), (\psi' \cup \psi)) \longrightarrow v' \text{ for some } v' \in \mathcal{V}[[S]] \} \\
 962
 \end{aligned}$$

963 *The well-foundedness of value typing relies on the fact that the global context parameter*
 964 *declaration is stratified, i.e., the type of the n -th context parameter may only refer to context*
 965 *parameters from 0 to $n - 1$. For example, the type of the very first context parameter must*
 966 *be a pure STLC type — it cannot refer to any context parameters (including itself).*

967 One might worry that context parameters with call-site binding could create cycles, thus
 968 breaking the well-foundedness. We consider two potential scenarios.

969 **Self-recursion.** With $p : \text{Int} \rightarrow \text{Int}$, one might attempt $(p \ 10)$ **with** $p = (\lambda x\{p\}.p \ x)$,
 970 where the lambda requires p at its call site rather than capturing it. If well typed, this would
 971 not terminate: calling p would look up p from the current environment, invoking the same
 972 closure indefinitely. However, this is ill-typed for one of two reasons: (1) if $p : \text{Int} \{\} \rightarrow \text{Int}$,
 973 then the lambda has type $\text{Int} \{p\} \rightarrow \text{Int}$, which does not match p 's declared type; (2) if one
 974 attempts $p : \text{Int} \{p\} \rightarrow \text{Int}$, this type is ill-formed because p is not in scope when checking its
 975 own type declaration.

976 **Mutual recursion.** With $p_1 : \text{Int} \{p_2\} \rightarrow \text{Int}$ and $p_2 : \text{Int} \{p_1\} \rightarrow \text{Int}$, the expression
 977 $(p_1 \ 10)$ **with** $p_2 = (\lambda x.p_1 \ x)$, $p_1 = (\lambda x.p_2 \ x)$ would not terminate. However, such mutual
 978 dependencies are prevented by the typing rule T-Prog: when checking the well-formedness
 979 of type T_i for context parameter p_i , only $p_0, \dots, p_{i-1}$ are in scope. Thus p_1 's type cannot
 980 mention p_2 while p_2 's type mentions p_1 —the declarations must be linearly ordered, preventing
 981 cycles.

982 ► **Definition 3** (Environment Compatibility). *We define compatibility relations between runtime*
 983 *environments and typing contexts:*

- 984 ■ $\rho \models_v \Gamma$ (normal environment satisfies variable context) iff for all $(x : T) \in \Gamma$, we have
 985 $x \in \text{dom}(\rho)$ and $\rho(x) \in \mathcal{V}[[T]]$.
- 986 ■ $\psi \models_p \Psi$ (context parameter environment satisfies parameter requirement) iff for all $p \in \Psi$
 987 and $p : T \in \Gamma$, we have $p \in \text{dom}(\psi)$ and $\psi(p) \in \mathcal{V}[[T]]$.

988 ► **Definition 4** (Full Environment Compatibility). *We write $\Gamma, \Psi \models (\rho, \psi)$ iff:*

- 989 ■ $\rho \models_v \Gamma$ (for variable bindings in Γ), and
- 990 ■ $\psi \models_p \Psi$ (for the global context parameter declarations).

991 A.2 Main Lemma

992 The main lemma states that well-typed terms, when evaluated in compatible environments
 993 that cover the required context parameters, terminate and produce values of the appropriate
 994 type.

995 ► **Lemma 5** (Main Lemma). *If $\Gamma \vdash t : T ! \Psi$ and $\Gamma, \Psi \models (\rho, \psi)$, then $\llbracket t \rrbracket(\rho, \psi) \longrightarrow v$ for some*
 996 *$v \in \mathcal{V}[[T]]$.*

997 **Proof.** By induction on the typing derivation $\Gamma \vdash t : T ! \Psi$. In each case, we show that
 998 evaluation terminates and produces a value of the appropriate type.

999 A.2.0.1 Case T-Var:

1000 We have $\Gamma \vdash x : T ! \emptyset$ where $(x : T) \in \Gamma$.

1001 By the compatibility assumption $\Gamma, \emptyset \models (\rho, \psi)$, we have $x \in \text{dom}(\rho)$ and $\rho(x) \in \mathcal{V}[[T]]$.

1002 By rule E-Var, $\llbracket x \rrbracket(\rho, \psi) \longrightarrow \rho(x)$.

1003 Let $v = \rho(x)$. We have $v \in \mathcal{V}[[T]]$.

1004 A.2.0.2 Case T-Cvar:

1005 We have $\Gamma \vdash p : T ! \{p\}$ where $(p : T) \in \Gamma$.

1006 By the compatibility assumption $\Gamma, \{p\} \models (\rho, \psi)$, we have $\psi(p) \in \mathcal{V}[[T]]$.

1007 By rule E-CVar, $\llbracket p \rrbracket(\rho, \psi) \longrightarrow \psi(p)$.

1008 Let $v = \psi(p)$. We have $v \in \mathcal{V}[[T]]$.

1009 A.2.0.3 Case T-Abs:

1010 We have

$$1011 \frac{\Gamma \vdash T \quad \bar{p} \in \Gamma \quad (\Gamma, x : T) \vdash t : S ! \Psi}{\Gamma \vdash \lambda(x:T)\{\bar{p}\}.t : T\{\bar{p}\} \rightarrow S ! (\Psi \setminus \bar{p})}$$

1012 By rule E-Abs, evaluating the lambda produces the closure:

$$1013 v = \langle \lambda x:T.t, \rho, (\psi \setminus \bar{p}) \rangle$$

1014 We need to show $v \in \mathcal{V}[[T\{\bar{p}\} \rightarrow S]]$.

1015 Consider any $v_2 \in \mathcal{V}[[T]]$ and any ψ' such that $\psi' \models_p \bar{p}$.

1016 Let $\rho' = \rho \cup \{x \mapsto v_2\}$ and $\psi'' = \psi' \cup (\psi \setminus \bar{p})$.

1017 We need to show that $\llbracket t \rrbracket(\rho', \psi'') \longrightarrow v'$ for some $v' \in \mathcal{V}[[S]]$.

1018 We have:

1019 ■ $(\Gamma, x : T), \Psi \models (\rho', \psi'')$:

1020 ■ $\rho' \models_v (\Gamma, x : T)$ since $\rho \models_v \Gamma$ and $v_2 \in \mathcal{V}[[T]]$.

1021 ■ $\psi'' \models_p \Psi$: For context parameters in Ψ , if they are in $\bar{p}$, they are covered by ψ' with
 1022 correct types (since $\psi' \models_p \bar{p}$); otherwise they are in $\psi \setminus \bar{p}$ (preserved from original with
 1023 correct types).

1024 By the induction hypothesis on the premise $(\Gamma, x : T) \vdash t : S ! \Psi$, $\llbracket t \rrbracket(\rho', \psi'') \longrightarrow v'$ for
 1025 some $v' \in \mathcal{V}[[S]]$.

1026 Therefore, $v \in \mathcal{V}[[T\{\bar{p}\} \rightarrow S]]$.

1027 **A.2.0.4 Case T-App:**

1028 We have

$$1029 \frac{\Gamma \vdash t : S\{\bar{p}\} \rightarrow T ! \Psi \quad \Gamma \vdash u : S ! \Psi'}{\Gamma \vdash t u : T ! (\Psi \cup \Psi' \cup \bar{p})}$$

1030 By assumption, ψ covers $(\Psi \cup \Psi' \cup \bar{p})$, so ψ covers Ψ , ψ covers Ψ' , and ψ covers $\bar{p}$.1031 By the induction hypothesis on the first premise, there exists $v_1 \in \mathcal{V}[S\{\bar{p}\} \rightarrow T]$ such
1032 that $\llbracket t \rrbracket(\rho, \psi) \rightarrow v_1$. Since v_1 is a closure, we have $v_1 = \langle \lambda x:S.t', \rho', \psi' \rangle$ for some $x, t', \rho',$
1033 ψ' .1034 By the induction hypothesis on the second premise, there exists $v \in \mathcal{V}[S]$ such that
1035 $\llbracket u \rrbracket(\rho, \psi) \rightarrow v$.1036 By the definition of $\mathcal{V}[S\{\bar{p}\} \rightarrow T]$, since $v \in \mathcal{V}[S]$, $\psi \models_p \bar{p}$ (from compatibility assumption),
1037 the body evaluates: $\llbracket t' \rrbracket(\rho' \cup \{x \mapsto v\}, \psi \cup \psi') \rightarrow v'$ for some $v' \in \mathcal{V}[T]$.1038 By rule E-App, $\llbracket t u \rrbracket(\rho, \psi) \rightarrow v'$.1039 **A.2.0.5 Case T-Allow:**

1040 We have

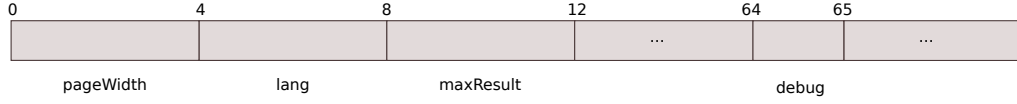
$$1041 \frac{\Gamma \vdash t : T ! \Psi \quad \bar{p} \in \Gamma \quad \Psi \subseteq \bar{p}}{\Gamma \vdash t \text{ allow } \bar{p} : T ! \bar{p}}$$

1042 By assumption, ψ covers $\bar{p}$.1043 Let $\psi' = \psi|_{\bar{p}}$. We have:1044 $\dashv$ $\Gamma, \bar{p} \models (\rho, \psi')$: The restriction only removes context parameters not in $\bar{p}$. Since all context
1045 parameters that might be used are in $\bar{p}$ (by $\Psi \subseteq \bar{p}$ and typing), compatibility is preserved.1046 By the induction hypothesis, there exists $v \in \mathcal{V}[T]$ such that $\llbracket t \rrbracket(\rho, \psi') \rightarrow v$.1047 By rule E-Allow, $\llbracket t \text{ allow } \bar{p} \rrbracket(\rho, \psi) \rightarrow v$.1048 **A.2.0.6 Case T-With:**

1049 We have

$$1050 \frac{(p : T) \in \Gamma \quad \Gamma \vdash u : T ! \Psi \quad \Gamma \vdash t : S ! \Psi'}{\Gamma \vdash t \text{ with } p = u : S ! (\Psi \cup (\Psi' \setminus \{p\}))}$$

1051 By assumption, ψ covers $(\Psi \cup (\Psi' \setminus \{p\}))$, so ψ covers Ψ and ψ covers $(\Psi' \setminus \{p\})$.1052 By the induction hypothesis on the typing of u , there exists $v \in \mathcal{V}[T]$ such that
1053 $\llbracket u \rrbracket(\rho, \psi) \rightarrow v$.1054 Let $\psi' = \psi[p \mapsto v]$. We have:1055 $\dashv$ $\Gamma, \Psi' \models (\rho, \psi')$: The binding $p \mapsto v$ is well-typed since $(p : T) \in \Gamma$ and $v \in \mathcal{V}[T]$.1056 $\dashv$ ψ' covers Ψ' : Since ψ covers $(\Psi' \setminus \{p\})$ and $p \in \text{dom}(\psi')$, we have ψ' covers Ψ' .1057 By the induction hypothesis on the typing of t , there exists $v' \in \mathcal{V}[S]$ such that
1058 $\llbracket t \rrbracket(\rho, \psi') \rightarrow v'$.1059 By rule E-With, $\llbracket t \text{ with } p = u \rrbracket(\rho, \psi) \rightarrow v'$.



■ **Figure 6** Virtual Addressing. The virtual address of a context parameter is relative to the beginning of the table.

A.2.0.7 Case T-Sub:

We have

$$\frac{\Gamma \vdash t : T ! \Psi \quad p \in \Gamma}{\Gamma \vdash t : T ! (\Psi \cup \{p\})}$$

By assumption, ψ covers $(\Psi \cup \{p\})$, so ψ covers Ψ .

By the induction hypothesis, there exists $v \in \mathcal{V}[[T]]$ such that $[[t]](\rho, \psi) \longrightarrow v$. ◀

A.3 Program Soundness

► **Corollary 6** (Program Soundness). *If $\Vdash (\overline{\text{param}p:T}, t) : S$, then $[[\overline{\text{param}p:T}, t]](\emptyset, \emptyset) \longrightarrow v$ for some $v \in \mathcal{V}[[S]]$.*

Proof. By rule T-Prog, we have $\Gamma = \overline{p:T}$ and $\Gamma \vdash t : S ! \emptyset$.

The typing judgment $\Gamma \vdash t : S ! \emptyset$ states that the term t has type S and requires *no* context parameters from the environment (the effect is $\emptyset$).

We verify the conditions of Lemma 5:

■ $\Gamma, \emptyset \models (\emptyset, \emptyset)$: The empty environments trivially satisfy the typing context for evaluation purposes, since the effect $\emptyset$ means no context parameters will actually be accessed.

By Lemma 5, $[[t]](\emptyset, \emptyset) \longrightarrow v$ for some $v \in \mathcal{V}[[S]]$.

By rule E-Prog, $[[\overline{\text{param}p:T}, t]](\emptyset, \emptyset) \longrightarrow v$. ◀

B Implementation

We intend context parameters to be widely used. Therefore, it is important to have efficient implementation for even the most performance-sensitive applications.

One idea is to pass context parameters as normal function parameters. However, that would incur non-trivial runtime overhead, so we do not pursue this approach. For single-threaded environments, the simplest solution is to use a global mutable store for each context parameter. In the following, we consider designs for more general scenarios.

Here we present two different solutions for the language runtime. One is based on *virtual addressing*, which has the *worst-case* complexity of $O(1)$ for reading and binding context parameters. The other is based on a *hash table*, which has an *average* complexity of $O(1)$ for reading and binding context parameters. For both cases, a fixed amount of memory is required per thread.

B.1 Virtual Addressing

Virtual addressing generates a layout for all context parameters in a compact flat array. Each context parameter is assigned a virtual address relative to the beginning of the array. This scheme is illustrated in Figure 6.

1092 B.1.0.1 Reading a context parameter

1093 To read a context parameter, it suffices to read at the offset of the array as indicated by
 1094 its virtual address. There is no need to worry about the presence/absence of the context
 1095 parameter: the type system guarantees that the to-be-read parameter must have a value.

1096 B.1.0.2 Binding a context parameter

1097 To bind a context parameter, it suffices to backup the existing value on the calling stack and
 1098 later restore the value: ⁸

```

1099 1 // Implementation for "expr with name = rhs"
1100 2 val value = rhs
1101 3
1102 4 val oldVal = readVal(addrForName) // backup old vlaue
1103 5 writeVal(addrForName, value)      // bind new value
1104 6 val res = expr
1105 7 writeVal(addrForName, oldVal)    // restore old value
1106 8
1107 9 res
1108
1109

```

1110 B.1.0.3 Pros and cons

1111 Virtual addressing does not require that the values have a uniform representation. For
 1112 example, in Figure 6, the integer `pageWidth` occupies four bytes of memory but the boolean
 1113 `debug` occupies only one byte. It has worst-case complexity $O(1)$, so it is ideal for performance-
 1114 sensitive applications. However, virtual addressing needs to assume a closed world in order
 1115 to generate a compact and flat layout for all context parameters. It may also consume more
 1116 memory than the implementation based on hash tables when only a fraction of the declared
 1117 context parameters are bound.

1118 B.2 Hash Table

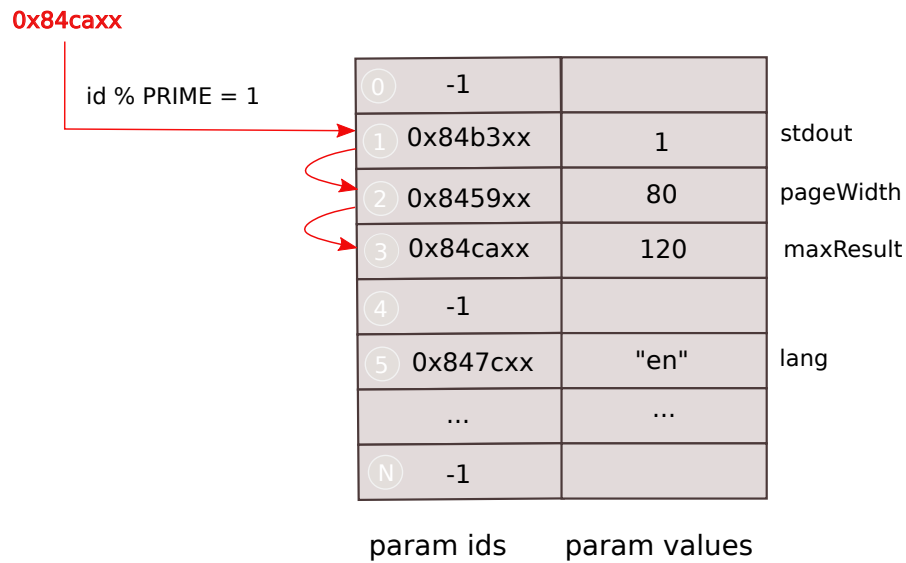
1119 The implementation based on a hash table is illustrated in Figure 7. It uses *linear probing* to
 1120 handle potential conflicts in hash keys. However, that particular choice is not important.
 1121 Any technique in the category of *open addressing* would be a good candidate because it can
 1122 benefit the memory locality of the hash table.

1123 The solution assumes that each context parameter has a unique and *non-negative* identifier.
 1124 For example, the unique identifier can be the *address* of the fully qualified name allocated
 1125 for a context parameter in the program area for constants.

1126 B.2.0.1 Reading a context parameter

1127 To read a context parameter, it suffices to compute its index in the hash table and read the
 1128 corresponding entry. Due to potential conflicts in hash value, the implementation needs to
 1129 check whether the actual unique identifiers match and follow the scheme used for resolving
 1130 conflicts. As guaranteed by the type system, it will never encounter the situation where the
 1131 to-be-read context parameter is missing.

⁸ For languages that support try/catch exceptions, it is necessary to wrap the evaluation of the inner expression in a try/finally block to ensure that the restoration code will be executed.



■ **Figure 7** Hash table with linear probing. Unused slots have the special value -1.

1132 B.2.0.2 Binding a context parameter

1133 Binding a context parameter involves backing up the existing key and value, evaluating the
 1134 expression, and finally restoring the original key and value. The implementation is illustrated
 1135 by the following pseudo code:

```

1136 1 // Implementation for "expr with name = rhs"
1137 2 val value = rhs
1138 3
1139 4 val index = computeIndex(idForName)
1140 5 val oldKey = readKeyAt(index) // backup old key and value
1141 6 val oldVal = readValAt(index)
1142 7
1143 8 writeKeyAt(index, idForName) // bind current value
1144 9 writeValAt(index, value)
1145 10 val res = expr
1146 11 writeKeyAt(index, oldKey) // restore old key and value
1147 12 writeValAt(index, oldVal)
1148 13
1149 14 res
1150
1151

```

1152 The reason that we need to also backup the key is due to the fact that the hash table
 1153 implementation depends on the special key -1 to indicate whether a slot is free or occupied,
 1154 to support linear probing.

1155 B.2.0.3 Pros and Cons

1156 The hashtable-based solution may save memory when only a fraction of context parameters
 1157 are active at the same time. The solution can work in an open-world environment, e.g.
 1158 dynamically linked libraries, as long as each context parameter has a unique identifier.

1159 One downside with the solution is that it requires values to have a uniform representation.
 1160 This is usually not the case in the runtimes of statically compiled languages, where values of
 1161 different types are represented using different amounts of memory. In that case, either boxing

1162 of primitive values is required or special support is required from the language runtime.

1163 **B.3 Thread Safety**

1164 In the presence of concurrency, we intend that the language runtime provides an isolated
1165 copy of the table for context parameters in thread-local storage. However, we have not
1166 implemented the design in our language nor assessed its pros and cons.

1167 An alternative approach is to use an immutable hash map as proposed by Steindorfer and
1168 Vinju [14]. This approach also eliminates the need to save and restore the existing bindings.
1169 The downside is that it needs additional memory allocation and collection, thus creating
1170 more pressure on the garbage collector. However, experiments are needed to show whether
1171 this is a valid concern in real-world workloads.

1172 More generally, concurrency support in a programming language is a big topic. We do
1173 not yet have a fully fleshed-out design for concurrency, which we leave to future work. We
1174 see both opportunities and challenges with the introduction of context parameters.

1175 With the elimination of global variables, it is possible to control the interference between
1176 threads in a local context. By controlling shared context parameters and local parameters, the
1177 programmer can narrow the surface of interference and thus reduce the chance of accidental
1178 data races. But there are open design questions: (1) what kind of concurrency primitive
1179 should the language provide; (2) how to control the sharing between threads; (3) how practical
1180 the proposed concurrency design is for real-world programming.